Fast and Optimal Extraction for Sparse Equality Graphs

ANONYMOUS AUTHOR(S)

Equality graphs (e-graphs) are used to compactly represent equivalence classes of terms in symbolic reasoning systems. Beyond their original roots in automated theorem proving, e-graphs have been used in a variety of applications. They have become particularly important as the key ingredient in the popular technique of *equality saturation*, which has notable applications in compiler optimization, program synthesis, program verification, and symbolic execution, among others. In a typical equality saturation workflow, an e-graph is used to store a large number of equalities that are generated by local rewrites during a saturation phase, after which an optimal term is *extracted* from the e-graph as the output of the technique. However, despite its crucial role in equality saturation, e-graph extraction has received relatively little attention in the literature, which we seek to start addressing in this paper. Extraction is a challenging problem and is notably known to be NP-hard in general, so current equality saturation tools rely either on slow optimal extraction algorithms based on integer linear programming (ILP) or on heuristics that may not always produce the optimal result. In fact, in this paper, we show that e-graph extraction is *hard to approximate* within any constant ratio. Thus, any such heuristic will produce wildly suboptimal results in the worst case. Fortunately, we show that the problem becomes tractable when the e-graph is sparse, which is the case in many practical applications. We present a novel parameterized algorithm for extracting optimal terms from e-graphs with low treewidth, a measure of how "tree-like" a graph is, and prove its correctness. We also present an efficient Rust implementation of our algorithm and evaluate it against ILP on a number of benchmarks extracted from the Cranelift benchmark suite, a real-world compiler optimization library based on equality saturation. Our algorithm optimally extracts e-graphs with treewidths of up to 10 in a fraction of the time taken by ILP. These results suggest that our algorithm can be a valuable tool for equality saturation users who need to extract optimal terms from sparse e-graphs.

CCS Concepts: • **Theory of computation → Equational logic and rewriting**.

Additional Key Words and Phrases: e-graphs, equality saturation, extraction, treewidth

ACM Reference Format:
Anonymous Author(s). 2024. Fast and Optimal Extraction for Sparse Equality Graphs. 1, 1 (April 2024), 24 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Equality saturation is a powerful symbolic reasoning technique that has gained significant attention in the programming languages, verification, automated reasoning and compiler optimization communities over the past decades [Joshi et al. 2002; Nandi et al. 2018, 2020; Premtoon et al. 2020; Stepp et al. 2011; Tate et al. 2009; Wang et al. 2020; Willsey et al. 2021; Zhang et al. 2023, 2022]. It is based on the older concept of an *e-graph* [Nelson and Oppen 1980; Nieuwenhuis and Oliveras 2005], a data structure that compactly encodes sets of equivalent expressions. By representing programs as e-graphs, equality saturation enables the exploration of numerous local transformations simultaneously. This approach was demonstrated to be effective in various domains, such as compiler optimization [Koehler et al. 2022; Laddad et al. 2023; Tate et al. 2009; Willsey

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2024/4-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

et al. 2021], code synthesis [Nandi et al. 2020, 2021], hardware design [Coward et al. 2023], testing [Arteaga et al. 2022], and verification [Garcia-Contreras et al. 2023; Stepp et al. 2011].

Equality Saturation Workflow. Typically, equality saturation starts from a simple e-graph representing the input program and then applies local rewritings over that e-graph over and over until a fixed point is reached or an optimization budget is exhausted, after which the e-graph represents superposed compositions of many equivalent subterms. The final step is to *extract* an actual program from the resulting entangled e-graph. We typically want to extract a program that is optimal or close to optimal according to some cost function, selecting the most beneficial set of transformed terms among all possibilities encoded in the e-graph.

Hardness of Extraction. The problem of e-graph extraction presents significant challenges [He et al. 2023; Rosenthal 2023]. However, much of the previous literature on equality saturation has focused on the process of rewriting and e-graph maintenance, with less attention given to the extraction phase, which is surprising given its central importance. This can be partially explained by the observation that extraction is NP-hard [Stepp 2011], which has led many authors and practitioners to rely either on off-the-shelf solvers, such as integer linear programming solvers, or on heuristic methods. Moreover, it turns out that extraction is even hard to approximate within any constant ratio unless P=NP, as we show later in this paper by reduction from the set cover problem [Karp 1972], thus inheriting its inapproximability [Alon et al. 2006; Dinur and Steurer 2014]. This means that any e-graph extraction heuristic will always yield results that are arbitrarily far from optimal on certain inputs. Consequently, most of the proposed equality saturation systems currently either yield suboptimal results, when they are based on heuristics, or require slow and expensive extraction phases, when they are based on optimal techniques like integer linear programming.

Sparsity and Treewidth. Many classical graph problems are NP-hard but can still be solved efficiently when the underlying graphs have certain structural sparsity properties. More specifically, the graphs produced by some problems exhibit low or even bounded *treewidth* and *pathwidth* [Bodlaender 1997, 2005; Robertson and Seymour 1986], which are graph parameters that measure respectively how *tree-like* and *path-like* a graph is. If the real-world instances of the problem are tree-like or path-like, we can often obtain efficient polynomial-time algorithms for them by computing their respective *tree* or *path decompositions* [Cygan et al. 2015; Robertson and Seymour 1983, 1984] and running some polynomial algorithms, such as dynamic programming, on the results. Finding the treewidth or pathwidth of arbitrary graphs are NP-hard problems in general [Bodlaender et al. 1991], but optimal tree/path decompositions can still be computed efficiently when the treewidth or pathwidth is small [Bodlaender 1996; Bodlaender and Kloks 1996]. There are also many approximate solutions to tree/path decompositions [Bodlaender et al. 2016; Dell et al. 2018; Korhonen 2021; Reed 1992; van Dijk et al. 2006].

Existing Parameterized Algorithms. These sparsity properties are used in several graph problems occurring in Programming Languages, Verification and Compilers. For instance, the control-flow graphs of structured programs and the call graphs of real-world programs are known to have small treewidth and pathwidth [Conrado et al. 2023; Thorup 1998], which was exploited to find faster algorithms for a wide variety of program analysis and compiler optimization problems [Aiswarya 2022; Chatterjee et al. 2020; Chatterjee and Lacki 2013; de Oliveira Oliveira and Vadiee 2023; Fearnley and Schewe 2012; Ferrara et al. 2005; Goharshady and Zaher 2023; Krause 2013; Obdržálek 2003]. In this paper, we use the sparsity parameters of e-graphs to derive an efficient algorithm for the Optimal Extraction problem. To the best of our knowledge, this is the first work to consider and leverage graph sparsity parameters over e-graphs.

Fast and Optimal Extraction. We aim to show that when e-graphs are sparse, which is often the case in practice, Optimal Extraction is feasible and can be achieved efficiently. In particular,

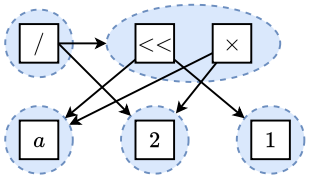


Fig. 1. An example e-graph, adapted from [Willsey et al. 2021].

we observed that the e-graphs produced by common equality saturation use cases typically have small *treewidth*. We thus propose the first parameterized algorithm for the optimal extraction of e-graphs, which takes time *linear* in the size of the e-graph when it has bounded treewidth/pathwidth (Theorems 4.1–4.2).

Implementation. We have implemented our algorithm in Rust using various low-level techniques, such as flattened tree representations, vertex coloring bitsets, and constant-time reference counting, to achieve high levels of performance. Although the implementation is not the main contribution of this paper, which we consider to be the algorithm itself and its theoretical properties, we provide an evaluation of our implementation on e-graphs taken from Cranelift [Fallin 2023], a popular equality saturation-based compiler backend, demonstrating that our approach drastically improves on the runtimes of integer linear programming, the only previous optimal extraction algorithm.

Contributions. Our specific contributions are as follows:

- We present some background on the challenges of optimal extraction of e-graphs for equality saturation, highlighting the need for a parameterized approach that can be applied in certain scenarios (section 2).
- We show that the problem of optimal extraction is hard to approximate by reduction from the set cover problem (section 3).
- We introduce a novel parameterized algorithm for the optimal extraction of e-graphs with bounded treewidth/pathwidth and prove its correctness and linear runtime (section 4).
- We provide an experimental evaluation of this algorithm on e-graphs taken from Cranelift (section 5).

2 PRELIMINARIES

In this section, we first define e-graphs and formalize the extraction problem we are considering. We then provide the required background from parameterized complexity including our graph sparsity parameters, namely treewidth and pathwidth.

2.1 Equality Graphs

E-Graphs. An *equality graph* (e-graph) is a data structure for compact representation of an equivalence relation. It was originally developed for use-cases in theorem-proving [Detlefs et al. 2005; Nelson and Oppen 1980], but later found many further applications including in program optimization and rewriting [Dershowitz 1993; Joshi et al. 2002], especially in the context of equality saturation [Nandi et al. 2020; Tate et al. 2011; Willsey et al. 2021]. It consists of two types of entities: *e-nodes*, which are our base objects/terms, and *e-classes*, which model the equivalence classes. Each e-class is a set of e-nodes which are considered equivalent objects/terms of the underlying language and each e-node has a list of children e-classes.

Example 1 ([Willsey et al. 2021]). *As a simple example, consider a compiler optimization scenario in which we want to optimally compile the expression* $(a \times 2)/2$. *Suppose that* a *is an unsigned integer*

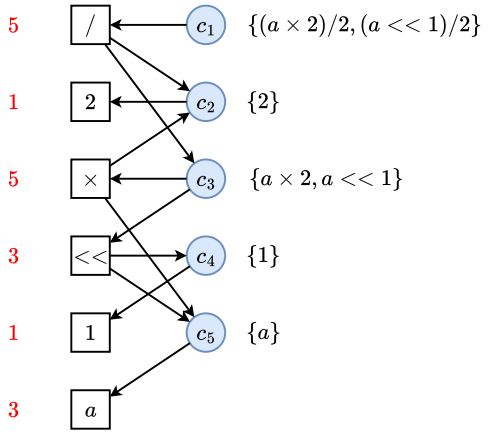


Fig. 2. Bipartite representation of the e-graph of Figure 1.

represented in binary and thus $a \times 2$ can equivalently be written as $a \ll 1$. The e-graph in Figure 1 models this scenario and would have typically been obtained by applying local rewritings to the e-graph of the original expression until saturation is reached. Here, each square vertex corresponds to an e-node and each ellipse to an e-class. Our goal is to compile the e-node labeled /. Each e-node has children (dependencies) which are e-classes. The different e-nodes in the same e-class are interchangeable. Thus, to compile /, we should first compile 2 and then either compile $\ll$ or $\times$. The choice of which to pick is based on a cost function that assigns a cost to each possible extraction. In this case, the cost of $\ll$ is lower than that of $\times$, so optimal extraction yields $(a \ll 1)/2$, which we pick for compilation.

Bipartite Representation. Although e-graphs are usually represented as in Figure 1, in this work we prefer an equivalent representation that includes vertices corresponding to both e-nodes and e-classes, as this will simplify our presentation. Thus, we formally define an e-graph as a directed *bipartite* graph $G = (C, \mathcal{N}, E)$ in which:

- C is a finite set of *e-classes*,
- $\mathcal{N}$ is a finite set of *e-nodes*, which is disjoint from C,
- $E \subseteq (C \times \mathcal{N}) \cup (\mathcal{N} \times C)$ is a set of directed edges.

Intuitively, every e-class has edges going to all of its member e-nodes, whereas every e-node has edges going to all of its children e-classes (dependencies). We write $V = C \cup \mathcal{N}$ as the set of vertices and $n := |V|$ to denote the number of vertices in the graph.

Example 2. *Figure 2 provides a bipartite representation of the e-graph of Figure 1. We introduce new vertices $c_1, \ldots, c_5$ to represent the original e-classes. Next to each e-class, we provide the set of expressions it models. In the rest of this paper, we only use the bipartite representation and all references to e-graphs are to bipartite representations of e-graphs.*

Saturation and Extraction. There are two fundamental tasks on e-graphs that are widely studied in the literature: (i) equality saturation, and (ii) extraction. The former focuses on extending the e-graph using rewrite rules or identifying that certain e-classes are equivalent. For example, egg [Willsey et al. 2021] can identify that c_1 and c_5 are equivalent in Figure 2 and thus merge them into a single e-class. The latter considers a given e-graph as input and attempts to extract the best way to compile a certain e-class or set of e-classes. For example, in Figure 2, to compile c_1, we might decide that it is less costly to compute it as $(a \ll 1)/2$ thus compiling $\{c_1, /, c_2, 2, c_3, \ll, c_4, 1\}$. The

two tasks are orthogonal in the sense that saturation tries to obtain an e-graph that encodes as much information as possible about the underlying optimization instance, thus giving us as many alternatives as possible for compiling the target term(s), whereas extraction focuses on finding the optimal solution in the resulting e-graph. In this work, we consider the problem of extraction.

Valid Extractions. Given an e-graph $G = (C, \mathcal{N}, E)$, with vertex set $V = C \cup \mathcal{N}$, a *valid extraction* or simply *extraction* of G is a set $\overline{V} \subseteq V$ of vertices such that:

- For every e-class $c \in \overline{V}$, there exists at least one e-node v such that $(c, v) \in E$ and $v \in \overline{V}$. In other words, if we extract an e-class, we have to extract at least one of its members (successor e-nodes).
- For every e-node $v \in \overline{V}$ and every e-class c such that $(v, c) \in E$, we have $c \in \overline{V}$. In other words, if we extract an e-node, we must also extract all of its children (successor e-classes).

Intuitively, if we consider the use-case in term-rewriting compiler optimizations, we would not need to evaluate more than one instance in each e-class since they are all equivalent, but if we choose to evaluate an e-node, we have to evaluate all of its dependencies/subterms*.

Monotone Cost Functions. A *cost function* is simply a function $\texttt{Cost} : 2^V \to \mathbb{R}$ that assigns a cost to every subset of vertices, including every extraction. In most use-cases, evaluating each e-node has an associated cost, e.g. the cost of performing $/, \times$ or $\ll$, and the total cost of an extraction is defined as the sum of costs of its e-nodes, with e-graphs being disregarded. However, we support a more expressive notion of cost function in our algorithms and can handle costs associated to both e-nodes and e-classes. Specifically, we say that a cost function is *monotone* if it satisfies the following two conditions:

- For all $v \in V$, we have $\texttt{Cost}(v) \geq \texttt{Cost}(\emptyset)$,
- For all $W_1, W_2 \subseteq V$ and $v \in V \setminus (W_1 \cup W_2)$, if we have $\texttt{Cost}(W_1) \leq \texttt{Cost}(W_2)$, then we also have $\texttt{Cost}(W_1 \cup \{v\}) \leq \texttt{Cost}(W_2 \cup \{v\})$.

In our algorithms, we assume that we are provided oracle access to the cost function. For simplicity, we present our algorithms over additive cost functions, i.e. cost functions in which $\texttt{Cost}(\emptyset) = 0$ and $\texttt{Cost}(\{v_1, v_2, \ldots, v_t\}) = \texttt{Cost}(\{v_1\}) + \texttt{Cost}(\{v_2\}) + \ldots + \texttt{Cost}(\{v_t\})$. We remark that this is only for ease of presentation and our algorithm applies to any monotone cost function.

Based on the discussion above, we are now ready to formally define our problem of Optimal Extraction for e-graphs.

Optimal Extractions. Given a set τ of target e-classes, a valid extraction $\overline{V} \supseteq \tau$ is optimal if for any other valid extraction $\overline{V}' \supseteq \tau$, we have $\texttt{Cost}(\overline{V}) \leq \texttt{Cost}(\overline{V}')$. Given an e-graph $G = (C, \mathcal{N}, E)$ and a target set $\tau \subseteq C$ as input and oracle access to a monotone cost function $\texttt{Cost}$, our goal is to output an optimal valid extraction $\overline{V}$.

Example 3. *On the Ethereum blockchain, users have to pay transaction fees (gas fees) for the execution of their smart contracts (programs) [Wood et al. 2014]. The gas fees are roughly proportional to the required amount of computation and cost more than 2 billion USD in 2023. Thus, optimizing them is an active area of research [Albert et al. 2021, 2023, 2022a,b]. The contracts are often implemented in a high-level programming language called Solidity [Ethereum Foundation 2024] and are then compiled to Ethereum virtual machine (EVM) bytecode. Each operation in this low-level language has a predefined gas cost. Consider an optimizing compiler that faces the expression $(a \times 2)/2$ in Solidity. It produces the e-graph of Figure 2 and would like to find the optimal extraction that includes $\tau = \{c_1\}$. Suppose that the cost of an extraction is the sum of costs of its e-nodes, which are shown in red next to each*

*In our figures, the choice of circles for e-classes and squares for e-nodes highlights this distinction.

e-node of Figure 2. In this case, the compiler can choose either of the following valid extractions:

$$\{c_1, /, c_2, 2, c_3, \times, c_5, a\} \quad \textit{Total cost: } 14$$
$$\{c_1, /, c_2, 2, c_3, \ll, c_5, a, c_4, 1\} \quad \textit{Total cost: } 13$$

An optimal extraction algorithm would find the latter, thus minimizing the gas costs.

2.2 Parameterized Complexity and Graph Sparsity

In this section, we provide a short overview of the basics of parameterized complexity and graph sparsity parameters. We refer to [Cygan et al. 2015] for a more detailed treatment. The fundamental idea in parameterized complexity is to analyze the runtime of an algorithm not only based on its input size n but also based on another parameter k. The parameter k can itself be part of the input or output or it may model some structural property of the instance. For example, in a graph problem, the diameter of the graph can be a parameter.

XP [Downey and Fellows 2012]. A problem is in the class XP if there is an algorithm that solves it in $O(n^{f(k)})$ time where f is an arbitrary computable function. XP stands for *piecewise-polynomial*. Intuitively, given an algorithm with such a runtime, we can solve the instances of the problem in which the parameter k is small in polynomial time. Thus, k can also be seen as a measure of hardness of the problem. Instances with small k are solvable in polynomial time whereas the general problem might not be.

FPT [Downey and Fellows 2012]. A problem is in the class *FPT (fixed-parameter tractable)* if it admits an algorithm with a runtime of $O(n^c \cdot f(k))$ where f is an arbitrary computable function and c is a constant not depending on either n or k. Intuitively, FPT problems can be solved in polynomial time on instances where the parameter k is small, but additionally, the degree of the polynomial is independent of k. In this work, we will provide linear-time FPT algorithms, i.e. we always have $c = 1$.

Graph Sparsity Parameters. As the name suggests, graph sparsity parameters are parameters that limit the number of edges in a graph and ensure that it is sparse. In this work, we will consider two classical graph sparsity parameters: treewidth [Robertson and Seymour 1984] and pathwidth [Robertson and Seymour 1983]. Informally speaking, treewidth measures the degree of tree-likeness of a graph. Trees and forests have a treewidth of 1 and graphs that resemble them have small treewidths. Intuitively, a graph with treewidth k can be decomposed into small parts, called bags, each of size at most $k + 1$, which are connected to each other in a tree-like manner. Similarly, pathwidth measures similarity to paths and a graph with pathwidth k can be decomposed into small parts of size at most $k + 1$ which are connected to each other like a path. We will formalize this further below. Any graph with treewidth/pathwidth k has $O(n \cdot k)$ edges.

Motivation for Sparsity Parameters. Many classical graph problems are NP-hard but nevertheless solvable in FPT when parameterized by treewidth/pathwidth [Bodlaender 1997, 2005; Robertson and Seymour 1986]. Thus, if the real-world instances of the problem are tree-like, then we have efficient polynomial-time algorithms for them. On the other hand, many widely-studied families of graphs such as cacti, series-parallel and outerplanar graphs, are known to have small treewidth [Bodlaender 1998]. Even when a problem is already in P, sparsity parameters can help reduce the runtime of the best known algorithms [Dong et al. 2021; Fomin et al. 2018].

Sparsity Parameters in Programming Languages, Verification and Compilers. Control-flow graphs of structured programs are known to have small treewidth and pathwidth [Conrado et al. 2023; Thorup 1998]. This sparsity has been exploited to find faster algorithms for a wide variety of problems in program analysis and compiler optimization, such as register allocation [Krause 2013], data-flow analysis [Chatterjee et al. 2020] and μ-calculus model checking [Obdržálek 2003].

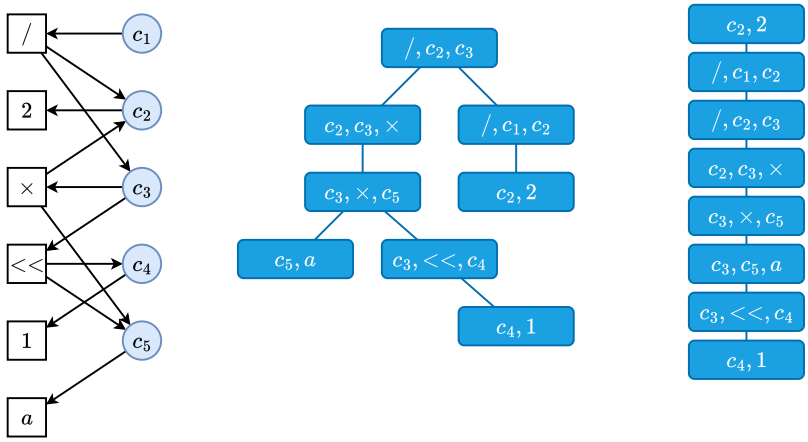


Fig. 3. The graph G of Figure 2 (left), a tree decomposition of G (center), and a path decomposition of G (right).

Moreover, call graphs of real-world programs have small treewidth/pathwidth, too [Goharshady and Zaher 2023]. A similar sparsity is also found in access graphs in the context of cache management [Ahmadi et al. 2022; Chatterjee et al. 2019]. However, to the best of our knowledge, this is the first work that considers graph sparsity parameters over e-graphs, as opposed to control-flow graphs or call graphs, and obtains fixed-parameter tractable algorithms for the optimal extraction problem.

Tree Decompositions [Cygan et al. 2015; Robertson and Seymour 1984]. Given a graph $G = (V, E)$, a *tree decomposition* of G is a tree $T = (\mathcal{B}, E_T)$ such that:

- Every node $b \in \mathcal{B}$ is called a *bag* and has an associated subset $V_b \subseteq V$ of the vertices of G. As is standard, we reserve the word vertex for the vertices of G and refer to the nodes of T as bags only.
- The bags cover all vertices of G. Formally, $\bigcup_{b \in \mathcal{B}} V_b = V$.
- The bags cover all edges of G. This means that for every edge, there is at least one bag that contains both of its endpoints. Formally, $\forall (u, v) \in E \quad \exists b \in \mathcal{B} \quad \{u, v\} \in V_b$.
- For every vertex $v \in V$, the set of bags containing v forms a connected subtree of T. Equivalently, if $b_1, b_2, b_3 \in \mathcal{B}$ and b_1 is on the unique path from b_2 to b_3 in T, then $V_{b_2} \cap V_{b_3} \subseteq V_{b_1}$.

Treewidth [Robertson and Seymour 1984]. The width of a tree decomposition is the size of its largest bag minus 1. Formally, $w(T) := \max_{b \in \mathcal{B}} |V_b| - 1$. The *treewidth* of the graph G is the smallest width among all of its tree decompositions. Thus, intuitively speaking, a graph with treewidth k can be decomposed into bags of size $k + 1$ or smaller that are connected to each other in a tree-like manner as in T.

Pathwidth [Cygan et al. 2015; Robertson and Seymour 1983]. A tree decomposition T is called a path decomposition if the underlying tree is a path. The *pathwidth* of G is the smallest width among its path decompositions. It is evident from the definitions that the treewidth of a graph is less than or equal to its pathwidth.

Example 4. *Figure 3 shows a tree decomposition and a path decomposition of the e-graph of Figure 2. Both decompositions have a width of* 2.

Cut Property [Cygan et al. 2015]. Let $T = (\mathcal{B}, E_T)$ be a tree decomposition of $G = (V, E)$ and ignore the edge directions in G. Consider two vertices $u, v \in V$ and two arbitrary bags b_u and b_v such that $u \in V_{b_u}$ and $v \in V_{b_v}$. If b is a bag on the unique path from b_u to b_v in T, then any path from u to v in G must intersect V_b. Similarly, if (b, b') is an edge of T on the unique path from b_u to b_v, then any path from u to v in G must intersect $V_b \cap V_{b'}$. Intuitively, every bag is a cut and so is every intersection of two neighboring bags. Thus, one can perform dynamic programming on tree decompositions in a manner quite similar to trees [Bodlaender 1988].

Computing Treewidth and Pathwidth. Finding the treewidth or pathwidth of an arbitrary graph are NP-hard problems [Bodlaender et al. 1991]. However, they are FPT with respect to the treewidth/pathwidth, i.e. there is an algorithm that given a graph G and a parameter k, decides in time $O(n \cdot f(k))$ if G has a treewidth/pathwidth of at most k and produces an optimal tree/path decomposition [Bodlaender 1996; Bodlaender and Kloks 1996]. There are also many other algorithms and tools providing both exact and approximate solutions and tree/path decompositions [Bodlaender et al. 2016; Dell et al. 2018; Korhonen 2021; Reed 1992; van Dijk et al. 2006]. Thus, without loss of generality, we assume that optimal tree/path decompositions are given as part of the inputs to our algorithms.

Complexity of Extraction. As we will see in Section 3, the problem of optimal extraction is NP-hard and hard-to-approximate even for very simple cost functions. Thus, unless P=NP, there is no hope that we can find a polynomial-time solution for the general case of this problem. However, as evidenced by our experiments in Section 5, in many real-world use-cases such as those of Cranelift [Fallin 2023], the e-graphs are quite sparse and have small treewidth/pathwidth. Thus, parameterized algorithms that exploit this sparsity would be able to solve a huge portion of real-world instances of e-graph extraction in polynomial time. In Section 4, we provide such algorithms. Our algorithms are linear-time FPT.

High-level Intuition. The high-level intuition behind our algorithms is based on the cut property above. Basically, in graphs of bounded treewidth/pathwidth, every bag of the decomposition is small. Moreover, every bag is a cut. Thus, we can successively find small cuts that break down the graph into smaller parts, enabling a simple and elegant dynamic programming algorithm that resembles bottom-up dynamic programming on trees/paths.

3 HARDNESS OF EXTRACTION

In this section, we prove that, unless P=NP, there is no polynomial-time algorithm to approximate the Optimal Extraction problem within any constant approximation factor. Our proof stands even for a simple cost function that assigns costs of 0 or 1 to all vertices.

Set Cover. Set Cover is one of the most famous combinatorial problems. Its decision variant was one of Karp's 21 NP-complete problems [1972]. The input consists of a universe $U = \{1, 2, \ldots, n\}$ and a collection $\mathcal{S}$ of m subsets $S_1, S_2, \ldots, S_m \subseteq U$. The task is to choose as few of the subsets as possible so that they cover the entire universe. More specifically, the goal is to find the smallest number t and $S_{i_1}, S_{i_2}, \ldots, S_{i_t} \in \mathcal{S}$ such that $\bigcup_{j=1}^{t} S_{i_j} = U$.

Hardness of Set Cover. While Set Cover is NP-hard, there is a simple greedy algorithm that provides an approximation ratio of $\ln n + 1$ [Chvatal 1979]. However, one cannot do much better than this approximation ratio as it is not possible to approximate Set Cover within a factor of $c \cdot \ln n$ for any $c < 1$ unless P=NP [Alon et al. 2006; Dinur and Steurer 2014].

Our Reduction. We provide a reduction from Set Cover to the Optimal Extraction problem over e-graphs. Consider an instance $\langle U = \{1, 2, \ldots, n\}, S = \{S_1, S_2, \ldots, S_m\}\rangle$ of Set Cover. We create an e-graph G as follows:

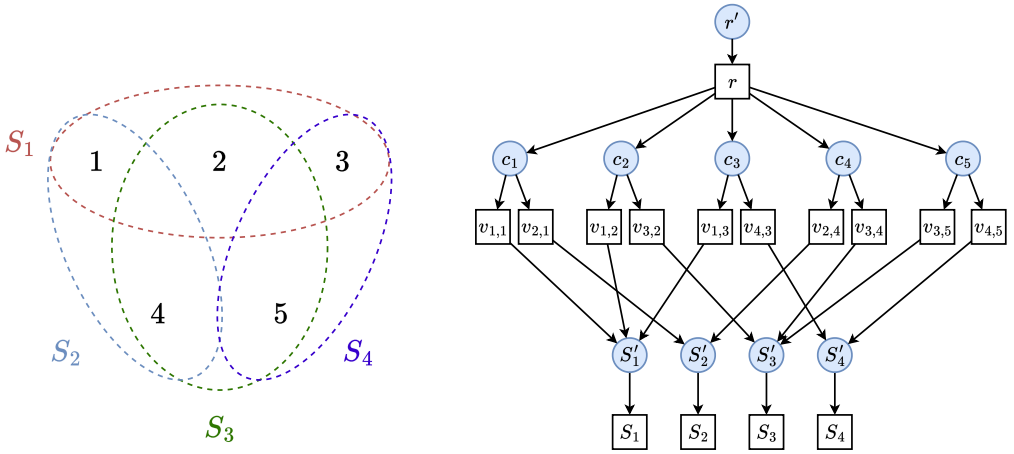


Fig. 4. A Set Cover instance (left) and its reduction to Optimal Extraction over an e-graph (right).

- For each set $S_i \in \mathcal{S}$, create an e-node S_i and an e-class S'_i. Add the edge (S'_i, S_i). Intuitively, we are creating an e-class S'_i that contains only the e-node S_i.
- For each universe element $j \in U$, create an e-class c_j.
- For each $S_i \in \mathcal{S}$ that contains j, add a new e-node $v_{i,j}$ to the e-class c_j, i.e. add the edge $(c_j, v_{i,j})$. Moreover, add the edge $(v_{i,j}, S'_i)$.
- Finally, create a root e-class r' and e-node r and add the edges (r', r) and (r, c_j) for each j. We set our target set $\tau = \{r'\}$.
- Assign a unit cost to every S_i and a cost of zero to every other vertex. The cost of a subset of vertices (an extraction) is defined simply as the sum of their individual costs.

Example 5. *Figure 4 provides an example Set Cover instance (left) and its reduction to Optimal Extraction over an e-graph (right).*

Correctness. We claim that the resulting e-graph has an extraction with cost t if and only if there are t subsets in the original Set Cover instance that cover the entire universe. Any extraction of the e-graph must contain r, r', every c_j and at least one successor $v_{i,j}$ of each c_j. However, if $v_{i,j}$ is included, this forces the inclusion of S'_i and thus S_i. Hence, the set of S_i's in any extraction cover the entire universe U. It is easy to verify that the converse holds, too. This reduction is polynomial-time.

The reduction above, together with the known hardness results on Set Cover, establish that Optimal Extraction over e-graphs cannot be approximated within any constant factor by a polynomial-time algorithm unless P=NP. This of course establishes NP-hardness, too. Thus, in the remainder of this work, we focus on designing parameterized algorithms based on the graph sparsity parameters treewidth and pathwidth.

4 OUR ALGORITHM

In this section, we provide our algorithm for Optimal Extraction. We assume that the input contains an e-graph $G = (\mathcal{C}, \mathcal{N}, E)$, a target set $\tau \subseteq V$ which must be present in every extraction and a tree/path decomposition $T = (\mathcal{B}, E_T)$ of G. We denote the number of vertices in G as n and the width of the decomposition T as k. Since an optimal decomposition can be computed in linear time, we assume without loss of generality that $|\mathcal{B}| \in O(n)$. For better presentation, we also assume

that the function Cost is additive. However, our algorithms are applicable to any monotone cost function.

Nice Decompositions [Cygan et al. 2015]. We say that a tree decomposition $T = (\mathcal{B}, E_T)$ rooted at the bag r is *nice* if it satisfies the following requirements:

- At the root bag r, we have $V_r = \emptyset$.
- For every leaf bag l, we similarly have $V_l = \emptyset$.
- Every non-leaf bag b is of one of the following types:
  - *Introduce Bag:* The bag b has a single child c and there is a single vertex $v \notin V_c$ such that $V_b = V_c \cup \{v\}$. In this case, we say that b *introduces* v.
  - *Forget Bag:* The bag b has a single child c and there is a single vertex $v \in V_c$ such that $V_b = V_c \setminus \{v\}$. We say that b *forgets* v.
  - *Join Bag:* The bag b has exactly two children c_1 and c_2 and we have $V_b = V_{c_1} = V_{c_2}$.

A nice path decomposition is defined analogously, except that it cannot contain join bags. Any tree/path decomposition can be converted to a nice decomposition in linear time by adding intermediate bags [Cygan et al. 2015]. Thus, without loss of generality, we assume that our decomposition T is nice. This will help simplify the cases we should consider in our algorithm below.

Example 6. *Figure 5 shows an example e-graph together with a tree decomposition and a nice tree decomposition. Leaf bags are shown in yellow, introduce bags in green, forget bags in red and join bags in purple.*

Subtree and Subgraph of a Bag. Consider a bag $b \in \mathcal{B}$. We define T_b as the subtree of our decomposition rooted at the bag b, i.e. the subtree containing b and its descendants. Each bag b has a set V_b of vertices. We define the set E_b of edges at b as those edges that have both their endpoints in V_b. More formally, $E_b = \{(u, v) \in E \mid u, v \in V_b\}$. Finally, we define the subgraph G_b as the part of the graph G that appears in T_b. More specifically,

$$G_b = \left(\bigcup_{b' \in T_b} V_{b'}, \bigcup_{b' \in T_b} E_{b'} \right).$$

By definition chasing, it is easy to verify that T_b is a valid decomposition for G_b. Moreover, the subgraphs G_b provide natural subproblems that can be considered in divide-and-conquer or dynamic programming approaches.

Dynamic Programming Subproblems. Our algorithm is a bottom-up dynamic programming on the decomposition T. Let $b \in \mathcal{B}$ be a bag and $M \subseteq V_b$ be a subset of vertices appearing in the bag b. Additionally, let $S \subseteq M \cap C$ be a subset of the e-classes in the bag b. We define a dynamic programming subproblem/variable $\texttt{Ans}[b, M, S]$ which is explicitly computed by our algorithm, as well as an implicit partial optimal solution $A[b, M, S]$. Our goal is to make sure that at the end of its computation, we have

$\texttt{Ans}[b, M, S] =$ the minimum cost of a subset $A[b, M, S]$ of vertices in G_b such that $A[b, M, S] \cap V_b = M$ and for every e-node $u \in A[b, M, S]$, every successor v of u in G_b is also in $A[b, M, S]$ and for every e-class $u \in A[b, M, S] \setminus S$, there exists a successor v of u in G_b such that $v \in A[b, M, S]$.

While this definition looks complicated, it is actually quite natural and intuitive: we define subproblems in which (i) we are restricting ourselves only to the part G_b of the e-graph that appears in the bag b and its descendants, (ii) we fix the set M of vertices in the current bag that are supposed to be in the partial solution (partial extraction) $A[b, M, S]$, (iii) we ensure that our partial solution satisfies the requirements of a valid extraction for e-nodes, meaning that if an e-node is included in

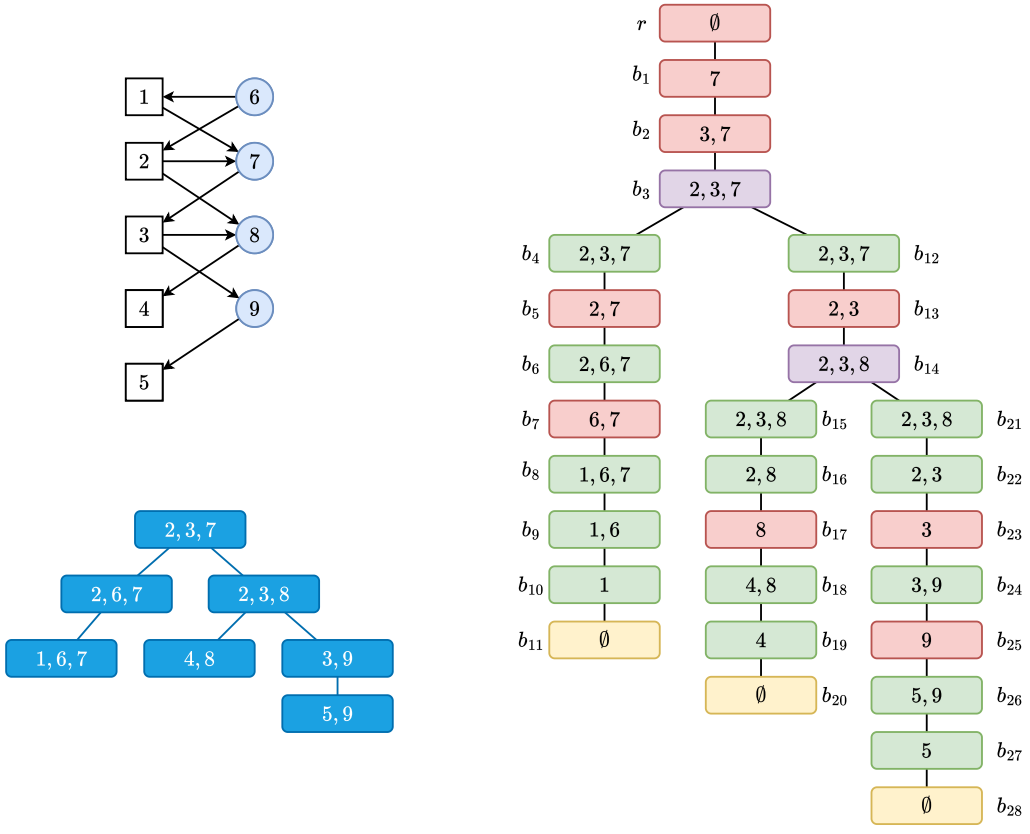


Fig. 5. An e-graph G (upper left corner), a tree decomposition of G (lower left corner) and a nice tree decomposition of G (right).

the solution, then all of its succcessor vertices must be included as well, and finally (iv) we ensure that our solution satisfies the extraction requirements on e-classes, meaning that each included e-class must have at least one successor in the solution. However, since this successor might come from outside of the current subgraph G_b, we also keep track of a set S of e-classes to whom we do not apply this requirement. We call these the *exempt* e-classes. Of course, setting S to $\emptyset$ would ensure that every included e-class has one of its successors in the solution.

Our Algorithm. Based on the intuition above, our algorithm processes the tree decomposition T in a bottom-up order and computes the $\texttt{Ans}[\cdot]$ values at each bag b using the following rules. We note that since our decomposition is nice, we have to consider only four types of bags. Moreover, the $A[\cdot]$ sets (optimal partial solutions) can be computed at the same time, too, or, as is standard in dynamic programming, by tracing the steps of the algorithm and remembering the optimal choices made at each point. Below, to save space, we assume that the $A[\cdot]$ sets are implicit and not computed by the algorithm. Thus, we only refer to them in establishing the correctness of our computation for $\texttt{Ans}[\cdot]$ values. The main procedure of our algorithm is shown in Pseudocode 1. It simply computes all the values in bottom-up order.

Sanity Checks. When computing $\texttt{Ans}[b, M, S]$ we first perform a local sanity check in bag b. Specifically, if there is an e-node $u \in M$ and one of its successors $v \in V_b \setminus M$, then it is impossible

Pseudocode 1 Our Main Procedure

```
1: function OptimalExtraction(G = (N, C, E), T = (B, E_T), r, τ, Cost)
2:     for all b ∈ B do
3:         if done[b] = false then
4:             Process(b)
5:     return Ans[r, ∅, ∅]
6: function Process(b)
7:     for all c ∈ children_T[b] do
8:         if done[c] = false then                ▷ Process children before parents
9:             Process(c)
10:    done[b] ← true
11:    for all M ⊆ V_b do
12:        for all S ⊆ M ∩ C do
13:            ComputeDP(b, M, S)
```

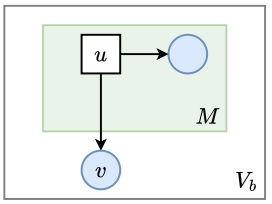


Fig. 6. A violation of our first sanity check, which is applied at every bag. If $u \in M$, but $v \notin M$, then there is no possible solution.

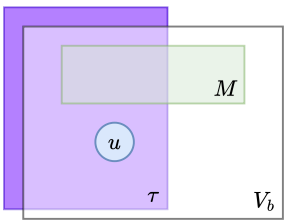


Fig. 7. A violation of our second sanity check, which is applied at every bag. We must have $\tau \cap V_b \subseteq M$.

to find a valid partial solution since we are forced by M to put u but not v into our solution, but u requires v. See Figure 6. Additionally, if there is a target vertex $u \in V_b \cap \tau$, then this vertex must be included in M. See Figure 7 and lines 2–7 of Pseudocode 2.

Example 7. *Consider the bag b_3 in Figure 5. This is a purple join bag and we have $V_{b_3} = \{2, 3, 7\}$. If we have $M = \{2, 3\}$ and $S = \emptyset$, then we are required to put 2 into the extraction, but exclude 7. However, this would lead to an invalid result since 2 is an e-node with an edge to 7. Thus, we set $\mathtt{Ans}[b_3, \{2, 3\}, \emptyset]$ to $+\infty$.*

Leaf Bags. If b is a leaf bag, then $V_b = \emptyset$ and the graph G_b is also empty. Thus, we set $\mathtt{Ans}[b, \emptyset, \emptyset] = \mathtt{Cost}(\emptyset)$. We note that the cost associated to the empty set is usually zero in practice, but our algorithm can handle any monotone cost function. We also have $A[b, \emptyset, \emptyset] = \emptyset$ as this is the

Pseudocode 2 Our Dynamic Programming

```
1: function ComputeDP(b, M, S)
2:     for all u ∈ M ∩ N do                                  ▷ Sanity Check for e-nodes
3:         for all v ∈ V_b such that (u, v) ∈ E do
4:             if v ∉ M then
5:                 return Ans[b, M, S] ← +∞
6:     if V_b ∩ τ ⊈ M then                                   ▷ Sanity check for target vertices
7:         return Ans[b, M, S] ← +∞
8:     if b is a leaf then
9:         Ans[b, M, S] ← Cost(∅)
10:    if b introduces u then
11:        c ← children_T[b][1]
12:        if u ∉ M then
13:            return Ans[b, M, S] ← Ans[c, M, S]
14:        if u ∈ C \ S ∧ M ∩ successors_G(u) = ∅ then
15:            return Ans[b, M, S] ← +∞
16:        O_{b,u} ← C ∩ predecessors_G(u)
17:        return Ans[b, M, S] ← Ans[c, M \ {u}, S ∪ O_{b,u} \ {u}] + Cost({u})
18:    if b forgets u then
19:        c ← children_T[b][1]
20:        return Ans[b, M, S] ← min{Ans[c, M, S], Ans[c, M ∪ {u}, S]}
21:    if b is a join bag then
22:        c_1, c_2 ← children_T[b][1], children_T[b][2]
23:        return Ans[b, M, S] ← min_{S_1∩S_2=S} (Ans[c_1, M, S] + Ans[c_2, M, S] − Cost(M))
```

only possible extraction, and thus the optimal extraction, for this subproblem. See lines 8-9 of Pseudocode 2.

Introduce Bags. This case is covered in lines 10-17 of Pseudocode 2. Suppose that b introduces the vertex u and has a single child c in T. We want to compute $\texttt{Ans}[b, M, S]$. First, if $u \notin M$, then any vertex that appears in the optimal solution $A[b, M, S]$ also appears in the previous optimal solution $A[c, M, S]$ at the child c. Thus, we have $A[b, M, S] = A[c, M, S]$ and $\texttt{Ans}[b, M, S] = \texttt{Ans}[c, M, S]$. See Figure 8.

All of u's successors in G_b must also be in V_b since u was just introduced and did not exist in G_c. Recall that every vertex appears in a connected subtree. Thus, if $u \in C \setminus S$ is an e-class, we check that M contains at least one successor of u. If not, we set $\texttt{Ans}[b, M, S] = +\infty$. See Figure 9.

Finally, if u is an e-node, the inclusion of the vertex u in the partial solution may satisfy the requirements of some e-classes. Let $O_{b,u}$ be the set of e-classes in M that have an edge to u. Since u is in the partial solution, these e-classes do not need another successor and can thus be exempt from this requirement in the rest of the graph. Hence, we have

$$A[b, M, S] = A[c, M \setminus \{u\}, S \cup O_{b,u} \setminus \{u\}] \cup \{u\}$$

and

$$\texttt{Ans}[b, M, S] = \texttt{Ans}[c, M \setminus \{u\}, S \cup O_{b,u} \setminus \{u\}] + \texttt{Cost}(\{u\}).$$

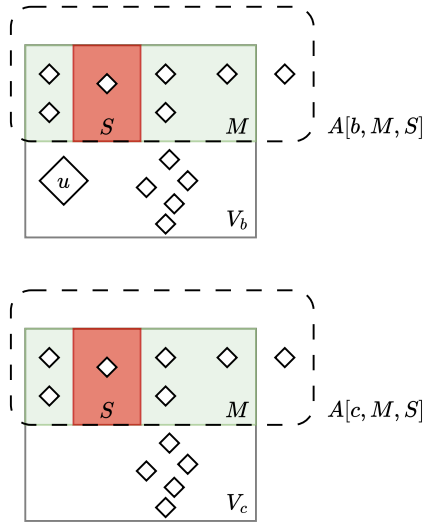


Fig. 8. Comparison of the partial solutions at an introduce bag b, introducing the vertex v, (top) and its child bag c (bottom) when $u \notin M$. Note that $S \subseteq M \subseteq V_b \cap V_c$. We use diamonds to denote vertices when we do not care if they are e-classes or e-nodes.

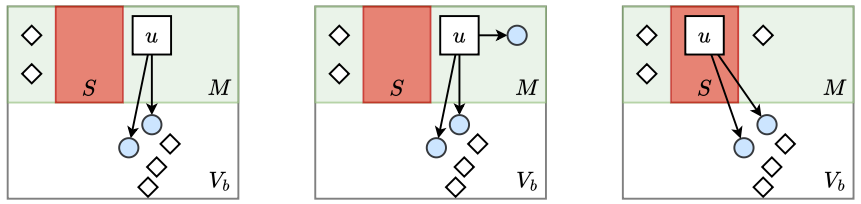


Fig. 9. The bag b introduces a new e-class u. The left pane is invalid since u does not have any successors in M. The middle pane is valid. The right pane is also fine since $u \in S$ is exempt.

See Figure 10. The same applies if u is an e-class in which case $O_{b,u} = \emptyset$. This is assuming an additive cost function. If `Cost` is not additive, we can instead set $\texttt{Ans}[b, M, S] = \texttt{Cost}(A[b, M, S])$ which is obtainable by one oracle call to `Cost`.

Example 8. *In Figure 5, the bag b_{18} introduces the e-class 8. Thus, we have $\texttt{Ans}[b_{18}, \{8\}, \emptyset] = +\infty$ because 8 is included in the partial solution but 4 is not. However, 8 needs at least one of its successors to be included. On the other hand, we will have $\texttt{Ans}[b_{18}, \{8\}, \{8\}] = \texttt{Ans}[b_{19}, \emptyset, \emptyset]$. In this case, since $8 \in S$, we are assuming that 8's requirements on its successors are to be satisfied by vertices outside of $G_{b_{18}}$. In other words, S is the set of e-classes that are exempt from the normal requirements. Now consider the bag b_4 that introduces the e-node 3. We have $\texttt{Ans}[b_4, \{2, 3, 7\}, \emptyset] = \texttt{Cost}(\{3\}) + \texttt{Ans}[b_5, \{2, 7\}, \{7\}]$. Here, 7 was added to the list of exempt vertices because its constraints are already satisfied by the inclusion of 3.*

Forget Bags. This case corresponds to Figure 11 and lines 18-20 of Pseudocode 2. Let b be a bag that forgets u and c be its only child. Since $V_b \subseteq V_c$, it is easy to verify that $G_b = G_c$. Thus, any partial solution found at c is also valid for b. The only subtle point here is that when computing $\texttt{Ans}[b, M, S]$, we should consider both M and $M \cup \{u\}$ at c since they both have the same intersection

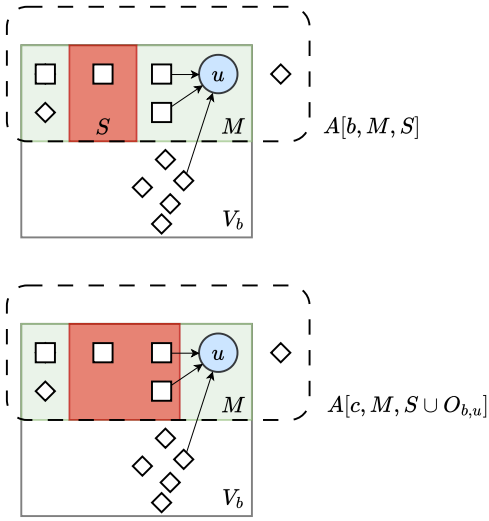


Fig. 10. The bag b introduces a new e-node u that is also in M. Thus, all predecessors of u have their requirements satisfied and can be added to the exemption set in the child bag c.

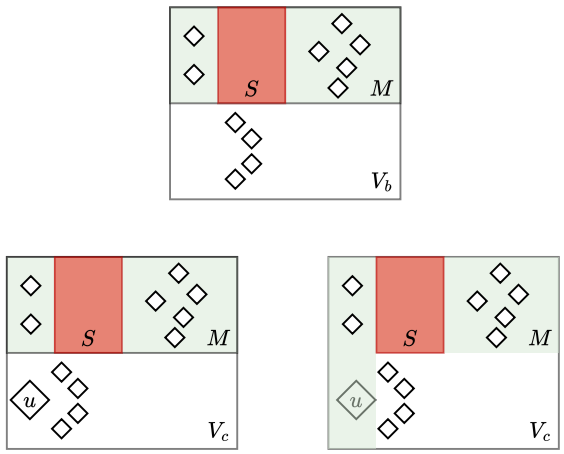


Fig. 11. The bag b forgets the vertex u. Thus, in its child c, we should consider both M and $M \cup \{u\}$.

M with V_b. Thus, we simply have

$$\mathsf{Ans}[b, M, S] = \min\{\mathsf{Ans}[c, M, S], \mathsf{Ans}[c, M \cup \{u\}, S]\}.$$

Example 9. *The bag b_{25} in Figure 5 forgets the e-node* 5. *Thus, we have*

$$\mathsf{Ans}[b_{25}, \{9\}, \emptyset] = \min\{\mathsf{Ans}[b_{26}, \{9\}, \emptyset], \mathsf{Ans}[b_{26}, \{5, 9\}, \emptyset]\}.$$

Join Bags. If b is a join bag, then it will have two children c_1 and c_2. Moreover, we have $V_b = V_{c_1} = V_{c_2}$ and $G_b = G_{c_1} \cup G_{c_2}$. Thus, the cost of the optimal solution in G_b is simply the sum of the costs in G_{c_1} and G_{c_2}, with two caveats:

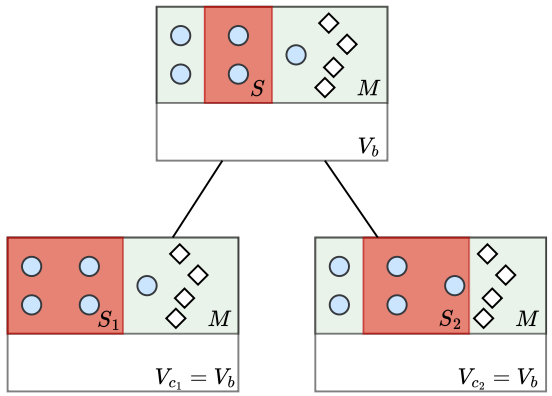


Fig. 12. The bag b joins its two children c_1 and c_2. The intersection of exemption sets in c_1 and c_2 should be the exemption set in b.

(1) We should not double-count vertices in $G_{c_1} \cap G_{c_2}$. However, the intersection of G_{c_1} and G_{c_2} is simply V_b and M already tells us which vertices in V_b are going to be in the solution.
(2) For every non-exempt e-class that is included in the extraction, we need not include two successors in both G_{c_1} and G_{c_2}, but only one of them. Thus, we can have different exemption sets S_1 and S_2, respectively at c_1 and c_2, as long as $S_1 \cap S_2 = S$. See Figure 12.

Therefore, we have

$$\texttt{Ans}[b, M, S] = \min_{S_1 \cap S_2 = S} \{\texttt{Ans}[c_1, M, S_1] + \texttt{Ans}[c_2, M, S_2] - \texttt{Cost}(M)\}.$$

This is applied in Lines 21-23 of Pseudocode 2. As in the case of introduce bags, the equation above assumes an additive cost function but can be replaced by oracle calls to `Cost`. Let S_1^*, S_2^* be the values of S_1 and S_2 that attained the minimum above, we have

$$A[b, M, S] = A[c_1, M, S_1^*] \cup A[c_2, M, S_2^*].$$

Example 10. *In Figure 5, the bag b_3 joins b_4 and b_{12}. Thus, we have*

$$\texttt{Ans}[b_3, \{3, 7\}, \{7\}] = \texttt{Ans}[b_4, \{3, 7\}, \{7\}] + \texttt{Ans}[b_{12}, \{3, 7\}, \{7\}] - \texttt{Cost}(\{3, 7\}).$$

The latter is subtracted because the costs of 3 and 7 were added in both subproblems. In this case, the only possible choice for S_1 and S_2 is to set them both to $\{7\}$.

Final Solution. Finally, when all the `Ans`$[\cdot]$ values are computed, the algorithm outputs `Ans`$[r, \emptyset, \emptyset]$. Recall that r is the root of our decomposition and we have $V_r = \emptyset$. Thus, `Ans`$[r, \emptyset, \emptyset]$ is by definition equal to the cost of the Optimal Extraction. As is standard in dynamic programming algorithms, the Optimal Extraction itself can be obtained by retracing the steps of the algorithm.

Theorem 4.1. *Given an e-graph $G = (\mathcal{N}, C, E)$ of size n, a rooted path decomposition $T = (\mathcal{B}, E)$ of G with $O(n)$ bags and width k, a set τ of target vertices and oracle access to a cost function* `Cost`*, our algorithm above computes the cost of an Optimal Extraction in G that includes all vertices in τ in time $O(n \cdot 3^k \cdot k^2)$.*

Proof. The algorithm's correctness is established by the discussion above for every type of bag. At each bag b, our algorithm considers all possible combinations of M and S. We note that $S \subseteq M \subseteq V_b$ and $|V_b| \leq k + 1$. Thus, our algorithm considers at most 3^{k+1} dynamic programming variables `Ans`$[\cdot]$ at each bag. This is because every vertex in V_b is either in both S and M or only in

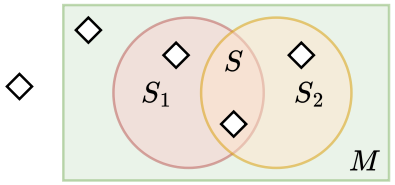


Fig. 13. A Venn diagram showing M, S, S_1, S_2 and the possible positions for a vertex $v \in V_b$.

M or in neither, for a total of three possibilities. Thus, we have $O(n \cdot 3^k)$ dynamic programming variables. By a straightforward case analysis in Pseudocode 2, it turns out that we spend at most $O(k^2)$ time computing each `Ans`$[\cdot]$ variable. Note that path decompositions do not have join nodes. Indeed, the most time-consuming step is the sanity check. Thus, our total runtime is $O(n \cdot 3^k \cdot k^2)$. □

Theorem 4.2. *Given an e-graph $G = (\mathcal{N}, C, E)$ of size n, a rooted tree decomposition $T = (\mathcal{B}, E)$ of G with $O(n)$ bags and width k, a set τ of target vertices and oracle access to a cost function* `Cost`*, our algorithm above computes the cost of an Optimal Extraction in G that includes all vertices in τ in time $O(n \cdot 5^k \cdot k)$.*

Proof. The only difference between nice tree decompositions and nice path decompositions is the presence of join bags. As argued above, our algorithm spends $O(n \cdot 3^k \cdot k^2)$ time on sanity checks and all other types of bags. Now consider the total runtime spent on line 23 of Pseudocode 2, where the algorithm is handling join nodes. We spend $O(k)$ time on each combination of (b, M, S, S_1, S_2). There are $O(n)$ choices for b. We must have $S, S_1, S_2 \subseteq M$ and $S_1 \cap S_2 = S$. Consider any vertex $v \in V_b$, there are 5 possibilities for v as in Figure 13:

- $v \notin M \cup S_1 \cup S_2 \cup S$,
- $v \in M \ \wedge \ v \notin S_1 \cup S_2 \cup S$,
- $v \in M \cap S_1 \ \wedge \ v \notin S_2 \cup S$,
- $v \in M \cap S_2 \ \wedge \ v \notin S_1 \cup S$,
- $v \in M \cap S_1 \cap S_2 \cap S$.

Thus, there are at most 5^{k+1} combinations of (M, S, S_1, S_2) at every bag b. Therefore, our total runtime is $O(n \cdot 5^k \cdot k)$.

□

Corollary 4.3. *On sparse e-graphs whose treewidth/pathwidth k is bounded by a constant, the Optimal Extraction problem is solved by our algorithm above in linear time, i.e. $O(n)$. This is in contrast to the general (non-sparse) case of the problem which is NP-hard and hard-to-approximate unless P=NP as established in Section 3.*

5 IMPLEMENTATION AND EXPERIMENTAL RESULTS

We now present our implementation and experimental evaluation.

Cranelift. Cranelift [Bytecode Alliance 2024] is an open-source optimizing compiler backend with a focus on compilation speed, simplicity and verifiability. It uses acyclic e-graphs to avoid heuristic pass order or expensive fixed-point computations [Fallin 2023]. The extraction algorithm used in Cranelift is a heuristic that does not take into account the cost of shared nodes. Thus, it cannot be directly compared with our approach which performs Optimal Extraction.

Benchmarks. As Cranelift works with function data-flow graphs, and given the previous results showing that control-flow and data-flow graphs are often sparse [Goharshady and Zaher 2023;

Thorup 1998], it is natural to experiment with Cranelift and see if the e-graphs are also sparse, and to compare the performance of our approach to integer linear programming solvers on sparse graphs. For benchmarks, we follow Cranelift and use the WebAssembly files in sightglass[†], which contains binaries of some commonly used libraries. In the plots below, each data point corresponds to one extraction called by Cranelift, which makes several extraction calls with different target sets τ on each graph.

Implementation. We implemented our algorithm as well as the standard integer linear programming solution in Rust and modified the Cranelift library to compute the optimal e-graph extraction for function data-flow graphs. Our implementation uses the `htd` library [Abseher and Röbke 2019] to compute tree and path decompositions. Our implementation is modular and can be integrated with any other treewidth/pathwidth library. For integer linear programming, we used the `good_lp` library [Lojkine et al. 2024] with the default CBC [Forrest et al. 2023] integer linear programming solver backend. To the best of our knowledge, ILP is the only previous approach for optimal extraction and all other methods are heuristics with no guarantees of optimality.

Machine. All experimental results were obtained on an Intel Core i9 12900HK Machine (14 cores, 20 threads, 24 MB Cache) with 32 GB of RAM running NixOS.

Optimizations. We have applied the following optimizations in our implementation:

- We represent the nice decomposition as a linear list of enums according to the pre-order traversal of the tree. Each bag is stored as an enum, containing either the vertex being introduced or forgotten, or an offset to its right child.
- Instead of doing recursion, we do the dynamic programming from the end of the list, and store the intermediate results when a leaf bag is encountered. As the tree is stored in pre-order, we have the result for both subtrees when we process a join bag. This avoids the need for recursion, which may be problematic for large graphs. Also, this compact flattened representation is more cache-friendly.
- With a simple greedy algorithm, one can compute a $(k+1)$-coloring for graphs with treewidth k where vertices in the same bag have different colors. If we assume the treewidth is at most 31, this means that one can represent subsets of bags as 32-bit integers, and use bitwise operations for efficient set operations.
- To track the partial solutions during dynamic programming without making the algorithm accidentally quadratic, we store the solutions as linked lists with node sharing. The representation supports set union and adding a single element, and can be done in constant time. We use reference counting to clean up unused solutions.

Statistics. Tables 1–3 show the distribution of e-graph sizes, treewidths and pathwidths in our benchmarks. We observe that almost 99 percent of the e-graphs, including almost 94 percent of the e-graphs with more than 100 vertices, have a treewidth of at most 10. However, only about 23% of the e-graphs have a pathwidth of 10 or smaller. Thus, our treewidth-based algorithm is applicable to the vast majority of the benchmarks. On the other hand, while our pathwidth-based variant provides a better asymptotic runtime, it is applicable to fewer benchmarks in practice due to the rapid increase in their pathwidth.

[†] https://github.com/bytecodealliance/sightglass

Number of Vertices	1-10	11-50	51-100	101-500	501-1000
Number of Graphs	15413	10527	4529	8417	1840

Number of Vertices	1001-5000	5001-10000	10001-50000	>50001
Number of Graphs	981	211	51	2

Table 1. Size distribution of our benchmark e-graphs.

Treewidth	1	2	3	4	5	6	7	8	9	10	>10
Number of Graphs	21942	6987	4966	3785	1156	816	571	508	312	215	713

Table 2. Treewidth distribution of our benchmark e-graphs. The last column also includes instances over which `tdh` failed to produce a tree decomposition.

Treewidth	1	2	3	4	5	6	7	8	9	10	>10
Number of Graphs	2	1013	3321	3127	1065	736	561	437	312	215	713

Table 3. Treewidth distribution of our benchmark e-graphs, excluding graphs with up to 100 vertices. The last column also includes instances over which `tdh` failed to produce a tree decomposition.

Pathwidth	1	2	3	4	5	6	7	8	9	10	>10
Number of Graphs	0	0	1	80	1785	2811	2780	1003	512	607	32392

Table 4. Pathwidth distribution of our benchmark e-graphs. The last column also includes instances over which `tdh` failed to produce a path decomposition.

ILP vs Our Treewidth-based Algorithm. Figure 14 (left) compares the runtimes of our treewidth-based algorithm with integer linear programming over the instances that had a treewidth of at most 10. Our algorithm comfortably beats ILP in this case, obtaining better runtimes in 95.9% of the Optimal Extraction instances. In 80.2% of the instances, our approach is more than twice faster than ILP. In this figure, our runtime includes both the time used to obtain the decomposition and the runtime of our dynamic programming. However, we should note that the overall time is by far dominated by the `htd` library used to obtain the decompositions, which is not as optimized as our implementation. If we consider the decompositions as part of the input and only compare the runtime of our dynamic programming algorithm with ILP, then the difference is much more pronounced. In this case, we are faster in 99.9% of the instances and more than ten times faster in 97% of the instances. This comparison is shown in Figure 14 (right). Note that the figures are in logarithmic scale.

ILP vs Our Pathwidth-based Algorithm. Figure 15 (left) provides a comparison of our pathwidth-based algorithm's runtimes with those of ILP over the instances that had a pathwidth of at most 10. In this case, our algorithm obtains better runtimes in all instances. We are at least twice faster than ILP in 99.8% of the instances. Figure 15 (right) provides the same comparison without considering the time required to compute path decompositions. In this case, our algorithm is more than 10 times faster than ILP in 88.7% of the instances.

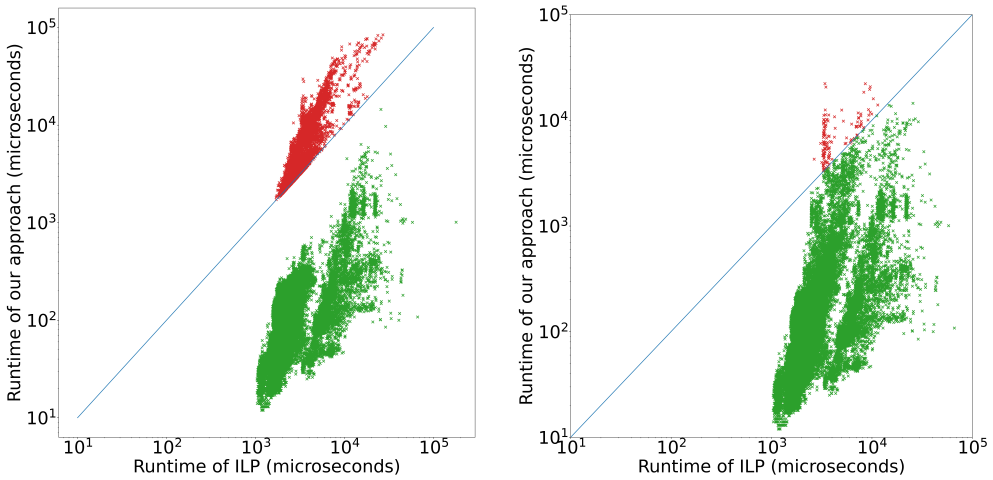


Fig. 14. Runtime comparison of our treewidth-based approach and integer linear programming for the Optimal Extraction problem. Each point corresponds to one extraction call made by Cranelift. Green points are the instances over which our approach was more efficient and red points are those for which ILP was faster. Both axes are in logarithmic scale. In the left figure, our reported runtime includes the time used by the library to compute decompositions but in the right figure, we only reported the runtime of our dynamic programming algorithm.

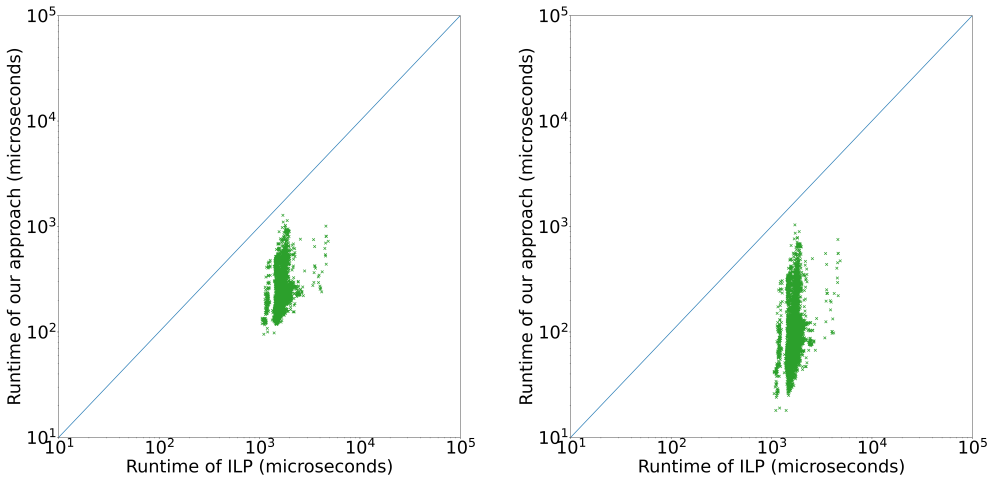


Fig. 15. Runtime comparison of our pathwidth-based approach and integer linear programming for the Optimal Extraction problem. Each point corresponds to one extraction call made by Cranelift. Green points are the instances over which our approach was more efficient and red points are those for which ILP was faster. Both axes are in logarithmic scale. In the left figure, our reported runtime includes the time used by the library to compute decompositions but in the right figure, we only reported the runtime of our dynamic programming algorithm.

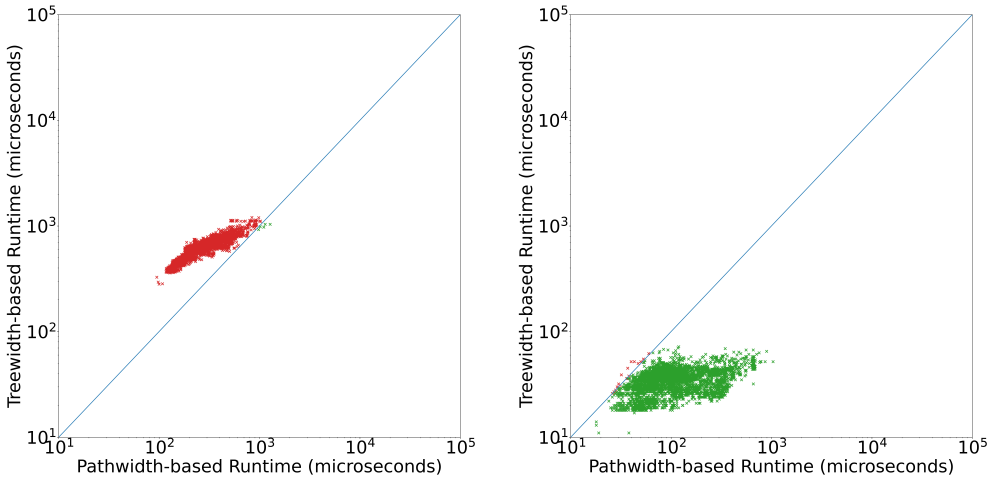


Fig. 16. Runtime comparison of the two variants of our algorithm. Each point corresponds to one extraction call made by Cranelift. Green points are the instances over which our treewidth-based approach was more efficient and red points are those for which the pathwidth-based variant was faster. Both axes are in logarithmic scale. In the left figure, our reported runtime includes the time used by the library to compute decompositions but in the right figure, we only reported the runtime of our dynamic programming algorithm.

Comparison of our Two Algorithms. Finally, Figure 16 provides a runtime comparison between the two variants (pathwidth-based and treewidth-based) of our algorithm over the instances to which they were both applicable, i.e. instances with pathwidth of at most 10. This figure showcases an interesting tradeoff. If we consider the total time, including the time to obtain decompositions, then the pathwidth-based approach is generally more efficient, but the situation reverses if we only consider the time needed for our dynamic programming. This might seem surprising since our dynamic programming's runtime dependence on pathwidth was much better than treewidth. However, note that pathwidth is often much larger than treewidth. We also observe that the `htd` library takes much more time to compute optimal tree decompositions in comparison with path decompositions.

Discussion. In summary, both variants of our algorithm, i.e. treewidth-based and pathwidth-based, comfortably beat the only previous solution for Optimal Extraction over sparse e-graphs, i.e. integer linear programming. Our treewidth-based algorithm is applicable to around 99% of the benchmark e-graphs, which have a treewidth of at most 10. In comparison, only about 23% of these e-graphs have a pathwidth of at most 10. However, when the pathwidth is small, the pathwidth-based variant is faster. There is therefore a tradeoff between applicability and efficiency.

6 CONCLUSION

In this work, we considered the problem of Optimal Extraction over e-graphs. The problem was known to be NP-hard, but we also showed that it is hard to approximate within any constant factor unless P=NP. We then observed that many real-world e-graphs are sparse and tree-like, and have a small treewidth/pathwidth. We exploited this fact to design a dynamic programming algorithm for Optimal Extraction that obtains a linear FPT runtime. Finally, we reported on an implementation of our approach and provided experimental results showcasing its applicability and significant gains in efficiency over the only previous optimal algorithm, i.e. integer linear programming, when the underlying e-graphs are sparse.

REFERENCES

Michael Abseher and Martin Röbke. 2019. htd: A small but efficient C++ library for computing (customized) tree and hypertree decompositions. https://github.com/mabseher/htd

Ali Ahmadi, Majid Daliri, Amir Kafshdar Goharshady, and Andreas Pavlogiannis. 2022. Efficient approximations for cache-conscious data placement. In *PLDI*. 857–871.

C. Aiswarya. 2022. How treewidth helps in verification. *ACM SIGLOG News* 9, 1 (2022), 6–21.

Elvira Albert, Jesús Correas, Pablo Gordillo, Guillermo Román-Díez, and Albert Rubio. 2021. *Don't run on fumes* - Parametric gas bounds for smart contracts. *J. Syst. Softw.* 176 (2021), 110923.

Elvira Albert, Samir Genaim, Daniel Kirchner, and Enrique Martin-Martin. 2023. Formally Verified EVM Block-Optimizations. In *CAV (3) (Lecture Notes in Computer Science, Vol. 13966)*. Springer, 176–189.

Elvira Albert, Pablo Gordillo, Alejandro Hernández-Cerezo, and Albert Rubio. 2022a. A Max-SMT Superoptimizer for EVM handling Memory and Storage. In *TACAS*, Vol. 13243. Springer, 201–219.

Elvira Albert, Pablo Gordillo, Alejandro Hernández-Cerezo, Albert Rubio, and Maria Anna Schett. 2022b. Super-optimization of Smart Contracts. *ACM Trans. Softw. Eng. Methodol.* 31, 4 (2022), 70:1–70:29.

Noga Alon, Dana Moshkovitz, and Shmuel Safra. 2006. Algorithmic construction of sets for k-restrictions. *ACM Trans. Algorithms* 2, 2 (2006), 153–177.

Javier Cabrera Arteaga, Nicholas Fitzgerald, Martin Monperrus, and Benoit Baudry. 2022. Wasm-mutate: Fuzzing WebAssembly compilers with e-graphs. In *E-Graph Research, Applications, Practices, and Human-factors Symposium*.

Hans L. Bodlaender. 1988. Dynamic Programming on Graphs with Bounded Treewidth. In *ICALP*, Vol. 317. 105–118.

Hans L. Bodlaender. 1996. A Linear-Time Algorithm for Finding Tree-Decompositions of Small Treewidth. *SIAM J. Comput.* 25, 6 (1996), 1305–1317.

Hans L Bodlaender. 1997. Treewidth: Algorithmic techniques and results. In *MFCS*. 19–36.

Hans L Bodlaender. 1998. A partial k-arboretum of graphs with bounded treewidth. *Theoretical computer science* 209, 1-2 (1998), 1–45.

Hans L Bodlaender. 2005. Discovering treewidth. In *International Conference on Current Trends in Theory and Practice of Computer Science*. 1–16.

Hans L. Bodlaender, Pål Grønås Drange, Markus S. Dregi, Fedor V. Fomin, Daniel Lokshtanov, and Michal Pilipczuk. 2016. A c^k n 5-Approximation Algorithm for Treewidth. *SIAM J. Comput.* 45, 2 (2016), 317–378.

Hans L. Bodlaender, John R. Gilbert, Ton Kloks, and Hjálmtyr Hafsteinsson. 1991. Approximating Treewidth, Pathwidth, and Minimum Elimination Tree Height. In *WG*, Vol. 570. 1–12.

Hans L. Bodlaender and Ton Kloks. 1996. Efficient and Constructive Algorithms for the Pathwidth and Treewidth of Graphs. *J. Algorithms* 21, 2 (1996), 358–402.

Bytecode Alliance. 2024. Cranelift: a fast, secure, relatively simple and innovative compiler backend. https://cranelift.dev/

Krishnendu Chatterjee, Amir Kafshdar Goharshady, Rasmus Ibsen-Jensen, and Andreas Pavlogiannis. 2020. Optimal and Perfectly Parallel Algorithms for On-demand Data-Flow Analysis. In *ESOP*, Vol. 12075. 112–140.

Krishnendu Chatterjee, Amir Kafshdar Goharshady, Nastaran Okati, and Andreas Pavlogiannis. 2019. Efficient parameterized algorithms for data packing. *Proc. ACM Program. Lang.* 3, POPL (2019), 53:1–53:28.

Krishnendu Chatterjee and Jakub Lacki. 2013. Faster Algorithms for Markov Decision Processes with Low Treewidth. In *CAV*, Vol. 8044. 543–558.

Vasek Chvatal. 1979. A greedy heuristic for the set-covering problem. *Mathematics of operations research* 4, 3 (1979), 233–235.

Giovanna Kobus Conrado, Amir Kafshdar Goharshady, and Chun Kit Lam. 2023. The Bounded Pathwidth of Control-Flow Graphs. *Proc. ACM Program. Lang.* 7, OOPSLA2 (2023), 292–317.

Samuel Coward, George A. Constantinides, and Theo Drane. 2023. Automating Constraint-Aware Datapath Optimization using E-Graphs. arXiv:2303.01839 [cs.AR]

Marek Cygan, Fedor V Fomin, Łukasz Kowalik, Daniel Lokshtanov, Dániel Marx, Marcin Pilipczuk, Michał Pilipczuk, and Saket Saurabh. 2015. *Parameterized algorithms*. Vol. 5.

Mateus de Oliveira Oliveira and Farhad Vadiee. 2023. From Width-Based Model Checking to Width-Based Automated Theorem Proving. In *AAAI*. 6297–6304.

Holger Dell, Christian Komusiewicz, Nimrod Talmon, and Mathias Weller. 2018. The PACE 2017 parameterized algorithms and computational experiments challenge: The second iteration. In *IPEC*.

Nachum Dershowitz. 1993. A Taste of Rewrite Systems. In *Functional Programming, Concurrency, Simulation and Automated Reasoning*. 199–228.

David Detlefs, Greg Nelson, and James B. Saxe. 2005. Simplify: a theorem prover for program checking. *J. ACM* 52, 3 (2005), 365–473.

Irit Dinur and David Steurer. 2014. Analytical approach to parallel repetition. In *STOC*. 624–633.

Sally Dong, Yin Tat Lee, and Guanghao Ye. 2021. A nearly-linear time algorithm for linear programs with small treewidth: a multiscale representation of robust central path. In *STOC*. 1784–1797.

Rodney G Downey and Michael Ralph Fellows. 2012. *Parameterized complexity*. Springer.

Ethereum Foundation. 2024. Solidity: An Object-Oriented High-level Language for Implementing Smart Contracts. https://docs.soliditylang.org/en/v0.8.25/

Chris Fallin. 2023. *Ægraphs: Acyclic E-graphs for Efficient Optimization in a Production Compiler (EGRAPHS 2023 - E-Graph Research, Applications, Practices, and Human-factors Symposium) - PLDI 2023*. https://cfallin.org/pubs/egraphs2023_aegraphs_slides.pdf

John Fearnley and Sven Schewe. 2012. Time and Parallelizability Results for Parity Games with Bounded Treewidth. In *ICALP*, Vol. 7392. 189–200.

Andrea Ferrara, Guoqiang Pan, and Moshe Y. Vardi. 2005. Treewidth in Verification: Local vs. Global. In *LPAR*, Vol. 3835. 489–503.

Fedor V Fomin, Daniel Lokshtanov, Saket Saurabh, Michał Pilipczuk, and Marcin Wrochna. 2018. Fully polynomial-time parameterized computations for graphs and matrices of low treewidth. *TALG* 14, 3 (2018), 1–45.

John Forrest, Ted Ralphs, Stefan Vigerske, Haroldo Gambini Santos, John Forrest, Lou Hafer, Bjarni Kristjansson, jpfasano, EdwinStraver, Miles Lubin, Jan-Willem, rlougee, jpgoncal1, Samuel Brito, useprefix=true family=gassmann, prefix=h-i, Cristina, Matthew Saltzman, tosttost, Bruno Pitrus, Fumiaki MATSUSHIMA, and useprefix=true family=st, prefix=to. 2023. *Coin-or/Cbc: Release Releases/2.10.11*. Zenodo. https://doi.org/10.5281/zenodo.10041724

Isabel Garcia-Contreras, Hari Govind V K, Sharon Shoham, and Arie Gurfinkel. 2023. Fast Approximations of Quantifier Elimination. arXiv:2306.10009 [cs.LO]

Amir Kafshdar Goharshady and Ahmed Khaled Zaher. 2023. Efficient Interprocedural Data-Flow Analysis Using Treedepth and Treewidth. In *VMCAI*, Vol. 13881. 177–202.

Mike He, Haichen Dong, Sharad Malik, and Aarti Gupta. 2023. Improving Term Extraction with Acyclic Constraints. (2023).

Rajeev Joshi, Greg Nelson, and Keith H. Randall. 2002. Denali: A Goal-directed Superoptimizer. In *PLDI*. 304–314.

Richard M. Karp. 1972. Reducibility Among Combinatorial Problems. In *Complexity of Computer Computations (The IBM Research Symposia Series)*. 85–103.

Thomas Koehler, Phil Trinder, and Michel Steuwer. 2022. Sketch-Guided Equality Saturation: Scaling Equality Saturation to Complex Optimizations of Functional Programs. arXiv:2111.13040 [cs.PL]

Tuukka Korhonen. 2021. A Single-Exponential Time 2-Approximation Algorithm for Treewidth. In *FOCS*. 184–192.

Philipp Klaus Krause. 2013. Optimal register allocation in polynomial time. In *CC*. 1–20.

Shadaj Laddad, Conor Power, Tyler Hou, Alvin Cheung, and Joseph M. Hellerstein. 2023. Optimizing Stateful Dataflow with Local Rewrites. arXiv:2306.10585 [cs.PL]

Ophir Lojkine et al. 2024. good-lp: Linear Programming for Rust, with a user-friendly API. https://github.com/rust-or/good_lp

Chandrakana Nandi, James R. Wilcox, Pavel Panchekha, Taylor Blau, Dan Grossman, and Zachary Tatlock. 2018. Functional programming for compiling and decompiling computer-aided design. *Proc. ACM Program. Lang.* 2, ICFP, Article 99 (jul 2018), 31 pages. https://doi.org/10.1145/3236794

Chandrakana Nandi, Max Willsey, Adam Anderson, James R. Wilcox, Eva Darulova, Dan Grossman, and Zachary Tatlock. 2020. Synthesizing structured CAD models with equality saturation and inverse transformations. In *PLDI*. ACM, 31–44.

Chandrakana Nandi, Max Willsey, Amy Zhu, Yisu Remy Wang, Brett Saiki, Adam Anderson, Adriana Schulz, Dan Grossman, and Zachary Tatlock. 2021. Rewrite rule inference using equality saturation. *Proc. ACM Program. Lang.* 5, OOPSLA, Article 119 (oct 2021), 28 pages. https://doi.org/10.1145/3485496

Greg Nelson and Derek C. Oppen. 1980. Fast Decision Procedures Based on Congruence Closure. *J. ACM* 27, 2 (1980), 356–364.

Robert Nieuwenhuis and Albert Oliveras. 2005. Proof-producing congruence closure. In *Proceedings of the 16th International Conference on Term Rewriting and Applications* (Nara, Japan) *(RTA'05)*. Springer-Verlag, Berlin, Heidelberg, 453–468. https://doi.org/10.1007/978-3-540-32033-3_33

Jan Obdržálek. 2003. Fast mu-calculus model checking when tree-width is bounded. In *CAV*. 80–92.

Varot Premtoon, James Koppel, and Armando Solar-Lezama. 2020. Semantic code search via equational reasoning. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation* (London, UK) *(PLDI 2020)*. Association for Computing Machinery, New York, NY, USA, 1066–1082. https://doi.org/10.1145/3385412.3386001

Bruce A. Reed. 1992. Finding Approximate Separators and Computing Tree Width Quickly. In *STOC*. 221–228.

Neil Robertson and Paul D Seymour. 1983. Graph minors. I. Excluding a forest. *Journal of Combinatorial Theory, Series B* 35, 1 (1983), 39–61.

Neil Robertson and Paul D. Seymour. 1984. Graph minors. III. Planar tree-width. *J. Comb. Theory, Ser. B* 36, 1 (1984), 49–64.

Neil Robertson and Paul D. Seymour. 1986. Graph minors. II. Algorithmic aspects of tree-width. *Journal of algorithms* 7, 3 (1986), 309–322.

Eli Rosenthal. 2023. E-graph Extraction Using ZDDs. In *E-Graph Research, Applications, Practices, and Human-factors Symposium*.

Michael Stepp. 2011. *Equality saturation : engineering challenges and applications*. Ph. D. Dissertation. University of California, San Diego, USA. http://www.escholarship.org/uc/item/85f640cc

Michael Stepp, Ross Tate, and Sorin Lerner. 2011. Equality-based translation validator for LLVM. In *Proceedings of the 23rd International Conference on Computer Aided Verification* (Snowbird, UT) *(CAV'11)*. Springer-Verlag, Berlin, Heidelberg, 737–742.

Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2009. Equality saturation: a new approach to optimization. In *Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Savannah, GA, USA) *(POPL '09)*. Association for Computing Machinery, New York, NY, USA, 264–276. https://doi.org/10.1145/1480881.1480915

Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. 2011. Equality Saturation: A New Approach to Optimization. *Log. Methods Comput. Sci.* 7, 1 (2011).

Mikkel Thorup. 1998. All structured programs have small tree width and good register allocation. *Information and Computation* 142, 2 (1998), 159–181.

Thomas van Dijk, Jan-Pieter van den Heuvel, and Wouter Slob. 2006. Computing treewidth with LibTW. (2006).

Yisu Remy Wang, Shana Hutchison, Jonathan Leang, Bill Howe, and Dan Suciu. 2020. SPORES: sum-product optimization via relational equality saturation for large scale linear algebra. *Proc. VLDB Endow.* 13, 12 (jul 2020), 1919–1932. https://doi.org/10.14778/3407790.3407799

Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.* 5, POPL (2021), 1–29.

Gavin Wood et al. 2014. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper* 2014 (2014), 1–32.

Yihong Zhang, Yisu Remy Wang, Oliver Flatt, David Cao, Philip Zucker, Eli Rosenthal, Zachary Tatlock, and Max Willsey. 2023. Better Together: Unifying Datalog and Equality Saturation. *Proc. ACM Program. Lang.* 7, PLDI, Article 125 (jun 2023), 25 pages. https://doi.org/10.1145/3591239

Yihong Zhang, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. 2022. Relational e-matching. *Proc. ACM Program. Lang.* 6, POPL, Article 35 (jan 2022), 22 pages. https://doi.org/10.1145/3498696